

《相机标定学习手册》

该内容是公众号 计算机视觉life 旗下课程 《单目/鱼眼/双目/全景/阵列 相机标定：原理和实战》 的课件编纂成的课程手册。

说明：本文会随课程持续更新，由于笔者水平有限，编写过程难免存在错误或疏漏，读者如发现有错误，欢迎提出。

扫描一下，系统学习视觉/激光/多传感器融合SLAM、三维重建等精品课程：



扫描学习SLAM、三维重建

第三章 非线性优化

1. 基础知识

- 1.1 范数
- 1.2 函数可微性
- 1.3 中值定理与泰勒(Taylor)展开
- 1.4 正定矩阵

2. 最小二乘问题

- 2.1 极值点以及必要条件
- 2.2 凸集和凸函数
- 2.3 非线性优化算法通用流程

3. 梯度下降法

- 3.1 下降方向
- 3.2 最速梯度下降法
- 3.3 线搜索方法
- 3.4. 牛顿法
- 3.5 高斯-牛顿法
- 3.6 LM (Levenberg-Marquardt) 算法

第三章 非线性优化

本章推荐参考文章 [METHODS FOR NON-LINEAR LEAST SQUARES PROBLEMS](#)

1. 基础知识

这里我们先介绍一些数学概念

1.1 范数

- l_1 范数: $\|x\|_1 = \sum_{i=1}^n |x_i|$
- l_2 范数: $\|x\|_2 = \sqrt{x^T x} = \sqrt{\sum_{i=1}^n x_i^2}$
- l_∞ 范数: $\|x\|_\infty = \max_{i=1}^n |x_i|$
- l_p 范数: $x_p = (\sum_{i=1}^n |x_i|^p)^{\frac{1}{p}}$

1.2 函数可微性

对于一个函数 $f: \mathcal{F} \subseteq \mathbb{R}^n$

- 连续: f 在每一点都连续
- 连续可微: 在每一点 $x \in \mathcal{F}$ 处, 每一个偏导数 $\frac{\partial f(x)}{\partial x_i}$ 存在且连续
- 二次连续可微: 在每一点 $x \in \mathcal{F}$ 处, 每一个偏导数 $\frac{\partial^2 f(x)}{\partial x_i \partial x_j}$ 存在且连续

如果函数为一阶连续可微的, 则

- **梯度(grad)**: $\nabla f(x) = (\frac{\partial f(x)}{\partial x_1}, \frac{\partial f(x)}{\partial x_2}, \dots, \frac{\partial f(x)}{\partial x_n})^T$
- **海森阵(Hessian)**, 其总为对称阵:

$$\nabla^2 f(x) = \begin{pmatrix} \frac{\partial^2 f(x)}{\partial x_1^2} & \frac{\partial^2 f(x)}{\partial x_1 \partial x_2} & \dots & \frac{\partial^2 f(x)}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f(x)}{\partial x_2 \partial x_1} & \frac{\partial^2 f(x)}{\partial x_2^2} & \dots & \frac{\partial^2 f(x)}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \dots & \vdots \\ \frac{\partial^2 f(x)}{\partial x_n \partial x_1} & \frac{\partial^2 f(x)}{\partial x_n \partial x_2} & \dots & \frac{\partial^2 f(x)}{\partial x_n^2} \end{pmatrix} = (\frac{\partial^2 f(x)}{\partial x_i \partial x_j})_{m \times n} \quad (1)$$

二次函数 $f(x) = \frac{1}{2} x^T A x + b^T x + c$, 其中 $A \in \mathbb{R}^{m \times n}, b \in \mathbb{R}^n, c \in \mathbb{R}$, 则有

$$\begin{aligned} \nabla f(x) &= Ax + b \\ \nabla^2 f(x) &= A \end{aligned} \quad (2)$$

- 若 F 为多变量矩阵, 则其一阶导**雅可比阵(Jacobi)**为

$$F'(x) = \begin{pmatrix} \frac{\partial F_1(x)}{\partial x_1} & \frac{\partial F_1(x)}{\partial x_2} & \dots & \frac{\partial F_1(x)}{\partial x_n} \\ \frac{\partial F_2(x)}{\partial x_1} & \frac{\partial F_2(x)}{\partial x_2} & \dots & \frac{\partial F_2(x)}{\partial x_n} \\ \vdots & \vdots & \dots & \vdots \\ \frac{\partial F_n(x)}{\partial x_1} & \frac{\partial F_n(x)}{\partial x_2} & \dots & \frac{\partial F_n(x)}{\partial x_n} \end{pmatrix} = (\frac{\partial F_i(x)}{\partial x_j})_{m \times n} \quad (3)$$

如多变量函数 $F(x) = Ax$ 则其Jacobi为 $F'(x) = A$

思考: 求函数 $f(x) = x_1^2 + 2x_2^2 - 2x_1x_2 - 4x_1 + 3$ 的梯度和Hessian阵

求函数 $F(x) = (e^{x_1x_2} + 3\sin x_2, 1 + x_2^2 \cos x_1)$ 的雅可比阵

1.3 中值定理与泰勒(Taylor)展开

设 $f: \mathcal{F} \Rightarrow \mathbb{R}$ 且 $x, x^* \in \mathcal{F}$, 若函数为一阶连续可微, 则

- 存在 $\alpha \in (0, 1)$ 使得 $f(x) = f(x^*) + \nabla f(\xi)^T(x - x^*)$, 其中 $\xi = x^* + \alpha(x - x^*)$
- f 在 x^* 处有一阶泰勒展开式 $f(x) = f(x^*) + \nabla f(x^*)^T(x - x^*) + o(\|x - x^*\|)$

如果函数为二阶可微的, 则有

- 存在 $\alpha \in (0, 1)$ 使得 $f(x) = f(x^*) + \nabla f(x^*)^T(x - x^*) + \frac{1}{2}(x - x^*)^T \nabla^2 f(\xi)(x - x^*)$, 其中 $\xi = x^* + \alpha(x - x^*)$

- f 在 x^* 处有二阶泰勒展开式

$$f(x) = f(x^*) + \nabla f(x^*)^T(x - x^*) + \frac{1}{2}(x - x^*)^T \nabla^2 f(\xi)(x - x^*) + o(\|x - x^*\|^2)$$

泰勒公式本质为使用某一点的各阶导数做系数构建一个多项式来近似函数在这一点邻域中的值。

1.4 正定矩阵

对于一个大小为 $n \times n$ 的**实对称矩阵** A 其为正定矩阵的定义为

$$x^T A x > 0, \forall x \neq 0 \quad (4)$$

如果

$$x^T A x \geq 0, \forall x \neq 0 \quad (5)$$

则称其为**半正定矩阵**，同样也可以定义**负定矩阵**，这里不在赘述

直观理解 上述表达式可以写为

$$\begin{aligned} A * X &= M \\ X^T * M &\geq 0 \\ \cos \theta &= \frac{X^T M}{\|X\| * \|Y\|} \end{aligned} \quad (6)$$

直觉理解为一个向量经过某个矩阵变换投影后，新的向量和其本身的夹角是小于等于 90° 的

其有几个重要的性质

1. 特征值均为非负
2. 顺序主子式均为非负
3. 存在秩为 r 的 $r \times n$ 实矩阵 B ，使 $A = B^T B$

2. 最小二乘问题

一般无约束优化问题

$$F(x) = \frac{1}{2} \sum_{i=1}^m (f_i(x))^2 = \frac{1}{2} \sum_{i=1}^m \|f(x)\|_2^2 \quad (7)$$

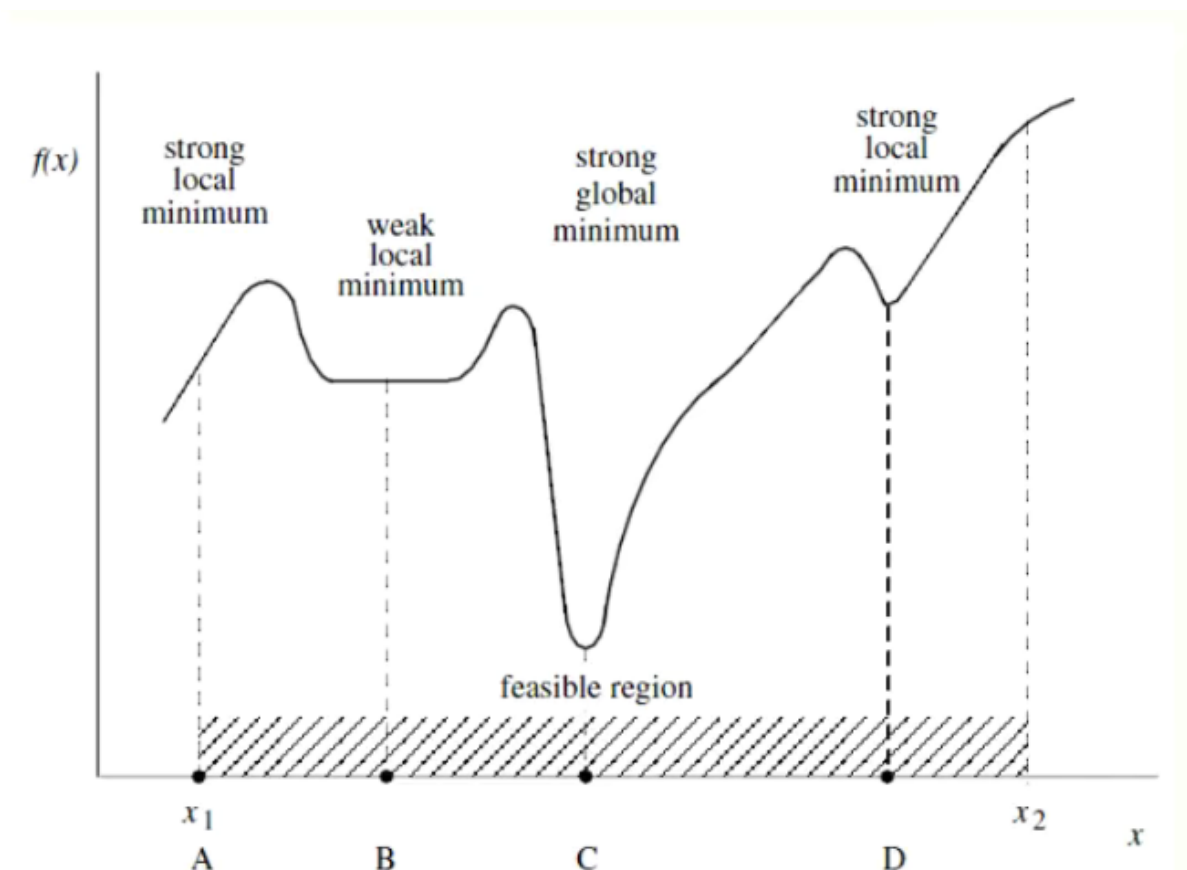
我们希望能够通过最小化 $F(x)$ 的方法，找到某个**给定形式的函数** $f_i(x)$ 的**系数**。

对应到相机标定，我们就是通过给定一些 3D 点和对应 2D 角点的坐标，使用给定的相机模型，通过最小化两者的重投影误差，来优化相机的参数。

问题转化为求解函数的对应极值点。

2.1 极值点以及必要条件

对应形如下图非线性的非凸函数



存在极小值

$$f(x) \geq f(x^*) \quad (8)$$

如果一个函数是一阶连续可导、或者二阶连续可导，其局部极小值存在的一阶必要条件为：

$$g(x^*)^T * d \geq 0 \quad (9)$$

我们这里简单推导一下，

对于 x^* 附近任意可行方向有，

$$x = x^* + \alpha d \quad (10)$$

使用一阶泰勒展开，有

$$f(x) = f(x^*) + \alpha g(x^*)^T d + o(\alpha \|d\|) \quad (11)$$

根据极小值定义，则有

$$f(x) - f(x^*) = \alpha g(x^*)^T d + o(\alpha \|d\|) \geq 0 \quad (12)$$

忽略掉余项，就有

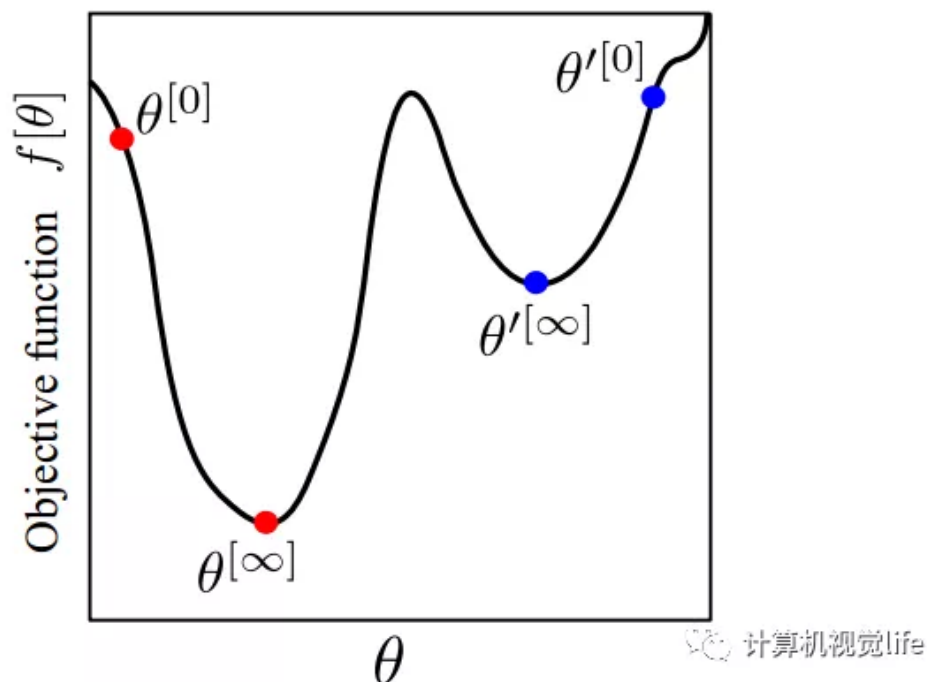
$$g(x^*)^T * d \geq 0 \quad (13)$$

当然，如果 d 的方向任意，那么就能得到

$$g(x^*)^T * d = 0 \quad (14)$$

这很容易理解，如果任意方向都满足，那么取 d 的负方向就可以证明

我们称满足 $g(x^*) = 0$ 的点为稳定点（驻点），其中包括了极大值点，极小值点和鞍点。



也存在二阶必要条件:

$$\begin{aligned} g(x^*)^T * d &\geq 0 \\ d^T H(x^*) d &\geq 0 \end{aligned} \quad (15)$$

我们只证明 $g(x^*) = 0$ 的情况

将函数进行二阶泰勒展开, 有

$$f(x) = f(x^*) + \alpha g(x^*)^T * d + \frac{1}{2} \alpha^2 d^T H(x^*) d + o(\alpha^2 \|d\|^2) \quad (16)$$

可以简单看到, 如果满足第一个公式, 则有

$$\frac{1}{2} \alpha^2 d^T H(x^*) d \geq 0 \quad (17)$$

也就是说 H 矩阵本身是**正半定**的, 对于极大值点, 其为**负半定**

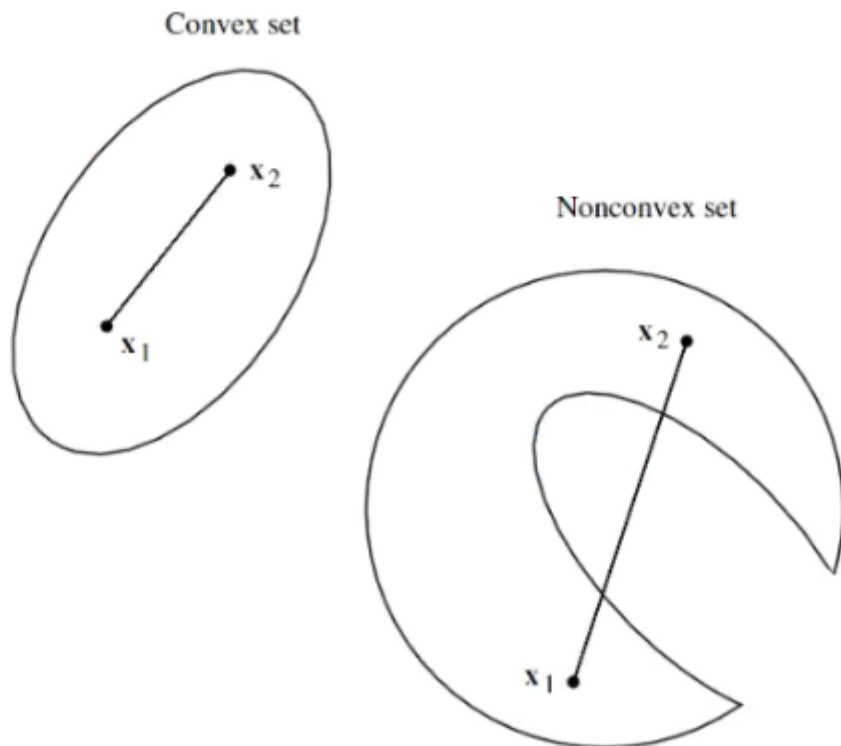
当然, $g(x^*) = 0$ 的时, 上述二阶条件其实也是局部极小值存在的充分条件。

2.2 凸集和凸函数

凸集: 给定一个非空集合 $F \subseteq R^n$, 如果 $\forall x, y \in F, \alpha \in [0, 1]$ 都有

$$\alpha x + (1 - \alpha)y \in F \quad (18)$$

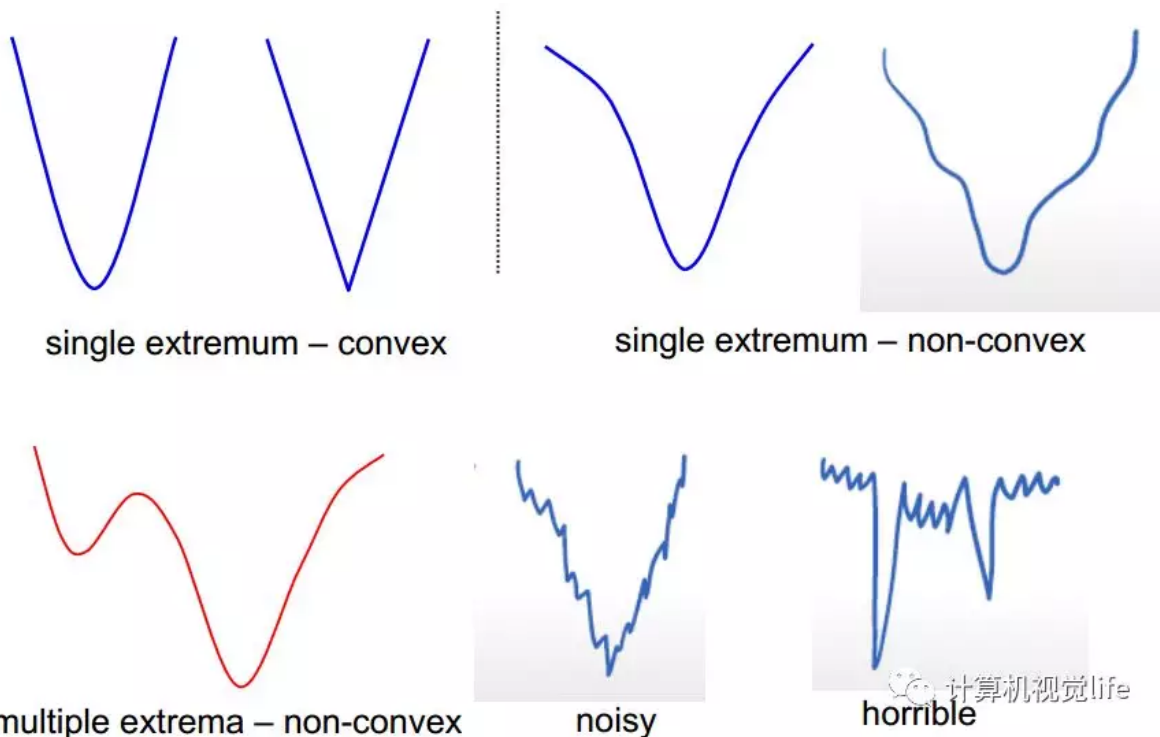
那么我们称该集合为凸集。



凸函数: 给定一个非空集合 $F \subseteq R^n$, 给定函数 $f: F \rightarrow R$ 如果满足

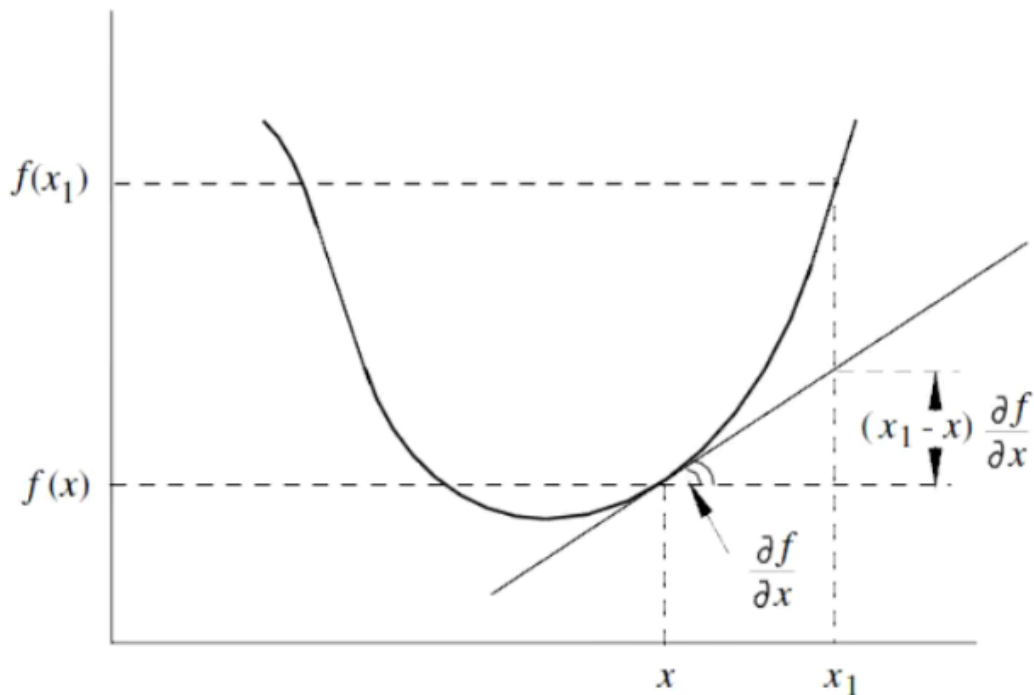
$$f(\alpha x + (1 - \alpha)y) \leq \alpha f(x) + (1 - \alpha)(f(y)) \quad (19)$$

则称函数为凸函数。



重要性质

$$\begin{aligned} f(x_1) &\geq f(x) + g(x)^T(x_1 - x) \\ \frac{1}{2}d^T H(x)d &\geq 0 \end{aligned} \quad (20)$$



也就是说其 H 矩阵为半正定的。

当然，当函数为凸函数时，上述一阶必要条件才是充分条件，即**凸函数有且只有一个极值点，且这个极值点是最值点**。

2.3 非线性优化算法通用流程

我们通常通过迭代的方法进行处理，基本流程可以总结如下(最小化问题)：

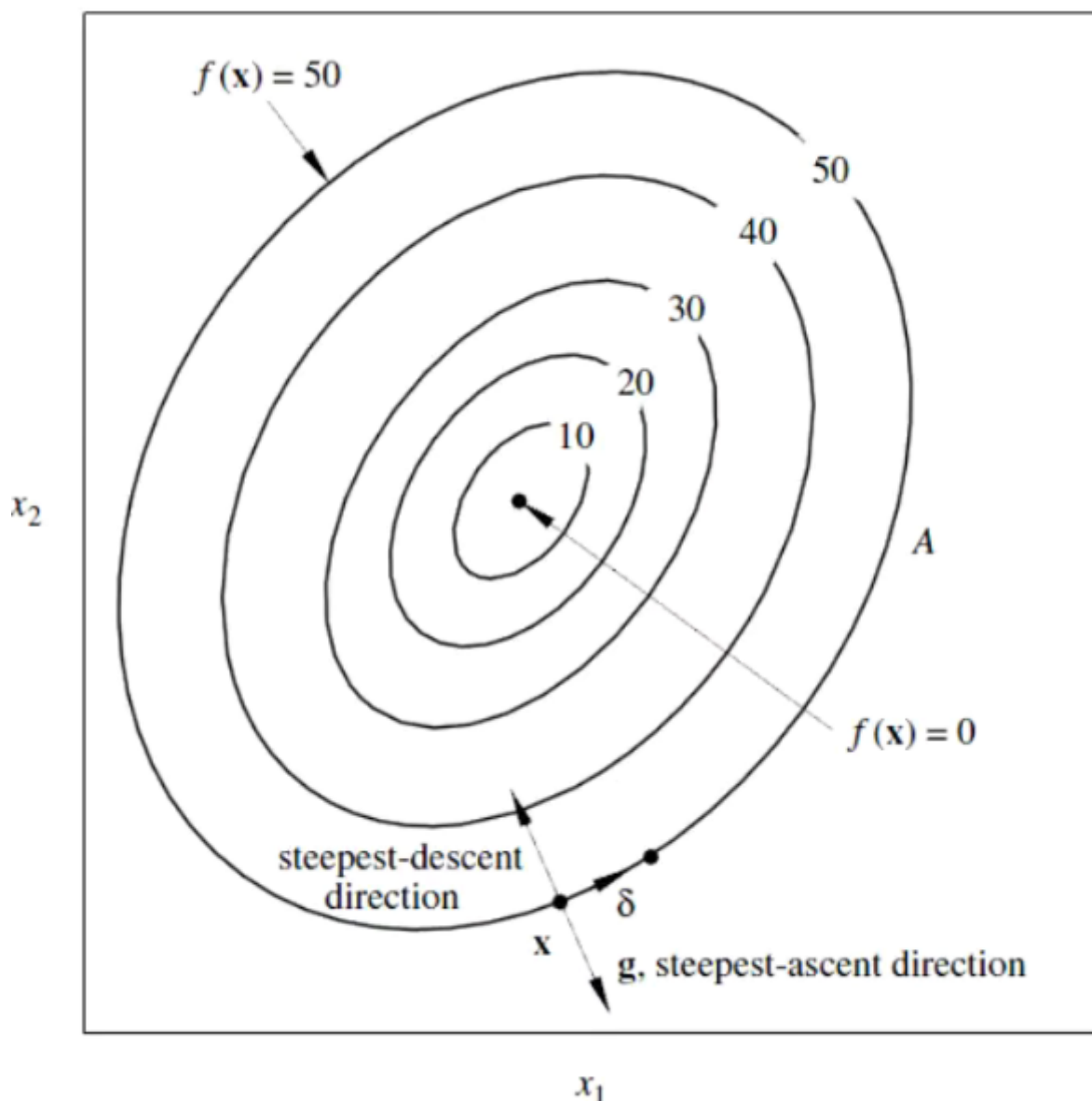
1. 给定某个初始值 x_0
2. 寻找增量 Δx_i ，使得 $f(x_{i+1}) \leq f(x_i)$
3. 更新变量 $x_{i+1} = x_i + \Delta x_i$ ，更新函数值 $f(x_{i+1})$
4. 如果达到某个条件（达到迭代次数或者函数值/ Δx_i 非常小），终止
5. 否则，返回第2步

可以看到，整个流程基本上就是在进行增量 Δx_i 的寻找，而其实不同的寻找方法和更新流程，就对应不同的优化方法。

为什么要用迭代的方法？因为梯度值为零的解并不好求。

3. 梯度下降法

3.1 下降方向



考虑一个方向 h 和该方向的足够小的步长 α ，通过泰勒展开我们得到

$$F(x + \alpha h) = F(x) + \alpha h^T g(x) + o(\alpha^2) \quad (21)$$

考虑到我们的目标是 $F(x + \alpha h) < F(x)$ ，那么得到如果方向满足

$$h^T g(x) = \|h\| \cdot \|g(x)\| \cdot \cos(\theta) < 0 \quad (22)$$

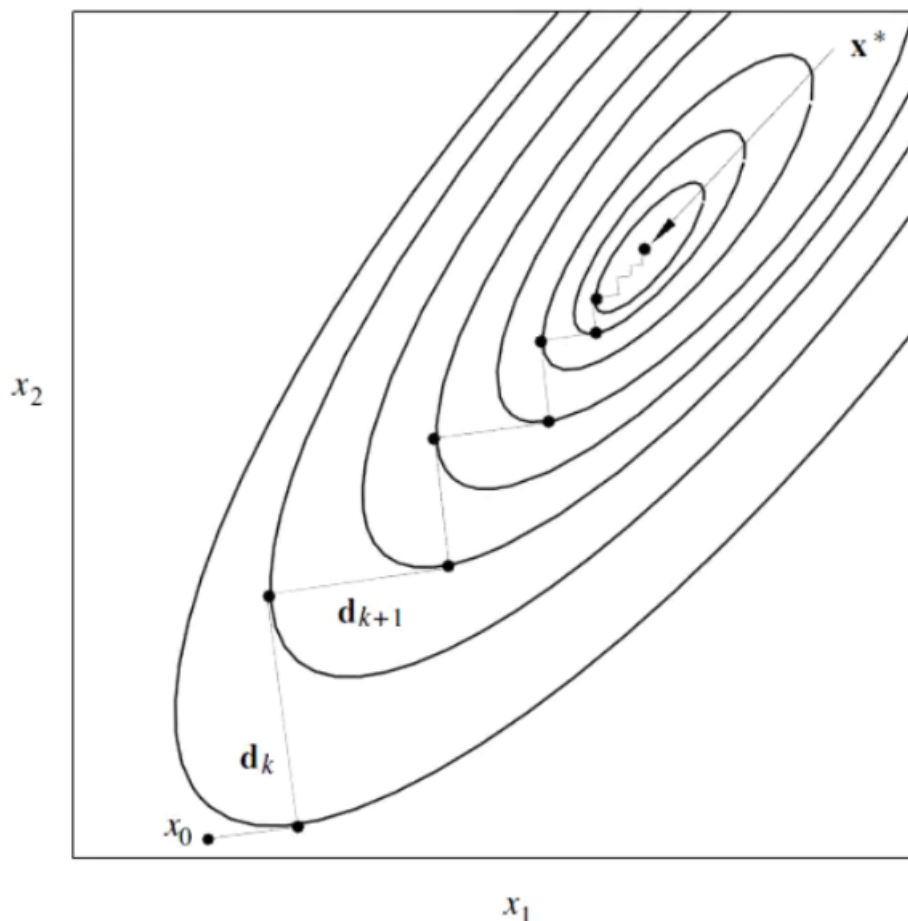
则 h 为 x 处的**下降方向**。

3.2 最速梯度下降法

当我们选取一个非负步长时，函数相对变化率为

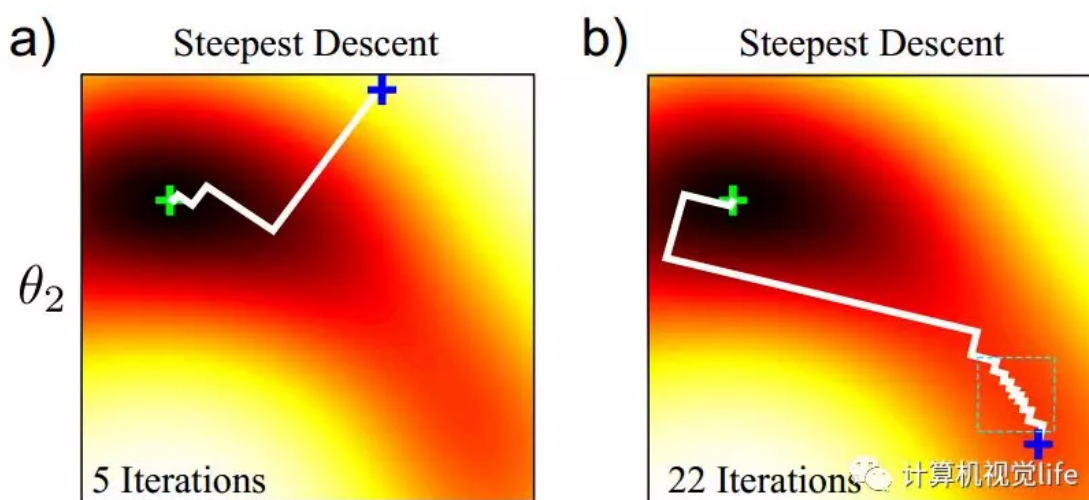
$$\lim_{\alpha \rightarrow 0} \frac{F(x) - F(x + \alpha h)}{\alpha \|h\|} = -\|g(x)\| \cdot \cos(\theta) \quad (23)$$

当 $\theta = \pi$ 时，我们有最大的下降值，而此时的下降方向就是**负梯度**方向。



梯度下降法存在的问题：

1. 初始值会影响迭代次数和结果



2. 我们只提了方向，那么步长该如何选择呢？

可以通过线性搜索的方法得到。

3.3 线搜索方法

我们知道，针对最速梯度下降法，我们不但需要知道变量的迭代方向，还需要知道变量更新的具体值是多少，我们定义

$$\varphi(\alpha) = F(x + \alpha h), x \text{ and } h \text{ fixed} \quad (24)$$

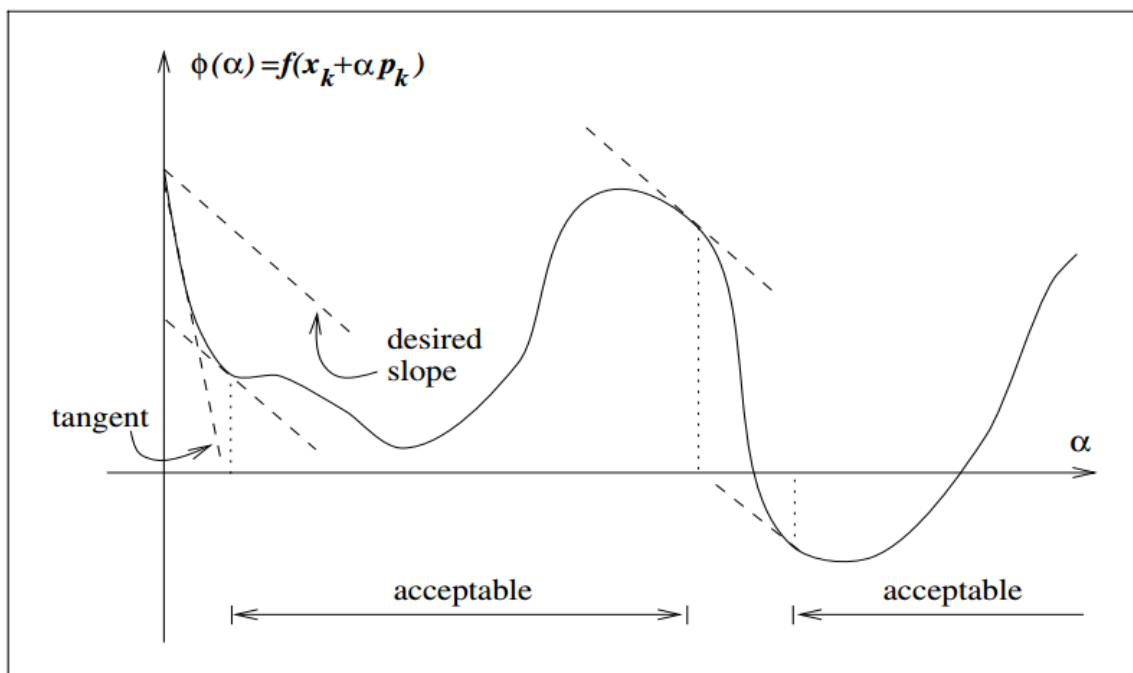


Figure 3.4 The curvature condition.

当给定一个初始的 α 时，上图给出了一种可能的函数形式，我们可以得到可能存在的几种状态：

1. α 过小，函数的变化值太小，这时我们需要增大 α
2. α 过大， $\varphi(\alpha) > \varphi(0)$ ，需要减小 α
3. α 接近最小值，接受

为了获得对应的值，我们有几种方法

- 精确线搜索：也使用近似迭代的方法来计算一个 α ，比如也可以使用牛顿法来计算最小值
- 非精确线搜索：实际使用中，我们没有必要找到确切的值，而是在某个范围达到一个可接受的值即可，考虑以下条件
 1. $\varphi(\alpha_s) \leq \varphi(0) + \gamma_1 * \varphi'(0) * \alpha$ 这里 $0 < \gamma_1 < 1$ ，这个条件保证了不会出现上述的状态2
 2. $\varphi'(\alpha_s) \geq \gamma_2 * \varphi'(0)$ ，这里 $\gamma_1 < \gamma_2 < 1$ ，这个条件保证了选取点不会离初始点太近，即避免了状态1

3.4. 牛顿法

我们来看牛顿法是怎么对最速下降法进行改进的，将函数进行二阶泰勒展开并忽略掉高阶项

$$F(x + h) = F(x) + g(x)h + h^T \frac{1}{2} H(x)h \quad (25)$$

我们知道要使该方程有极小值，对上式求导有

$$g(x + h) = g(x) + H(x) * h \quad (26)$$

令上式等于0

$$H(x) * h = -g(x) \quad (27)$$

当 H 为正定时，上式有唯一解 h ，有

$$0 < h^T H h = -h^T g(x) \quad (28)$$

说明该方向是一个下降方向，且其值为

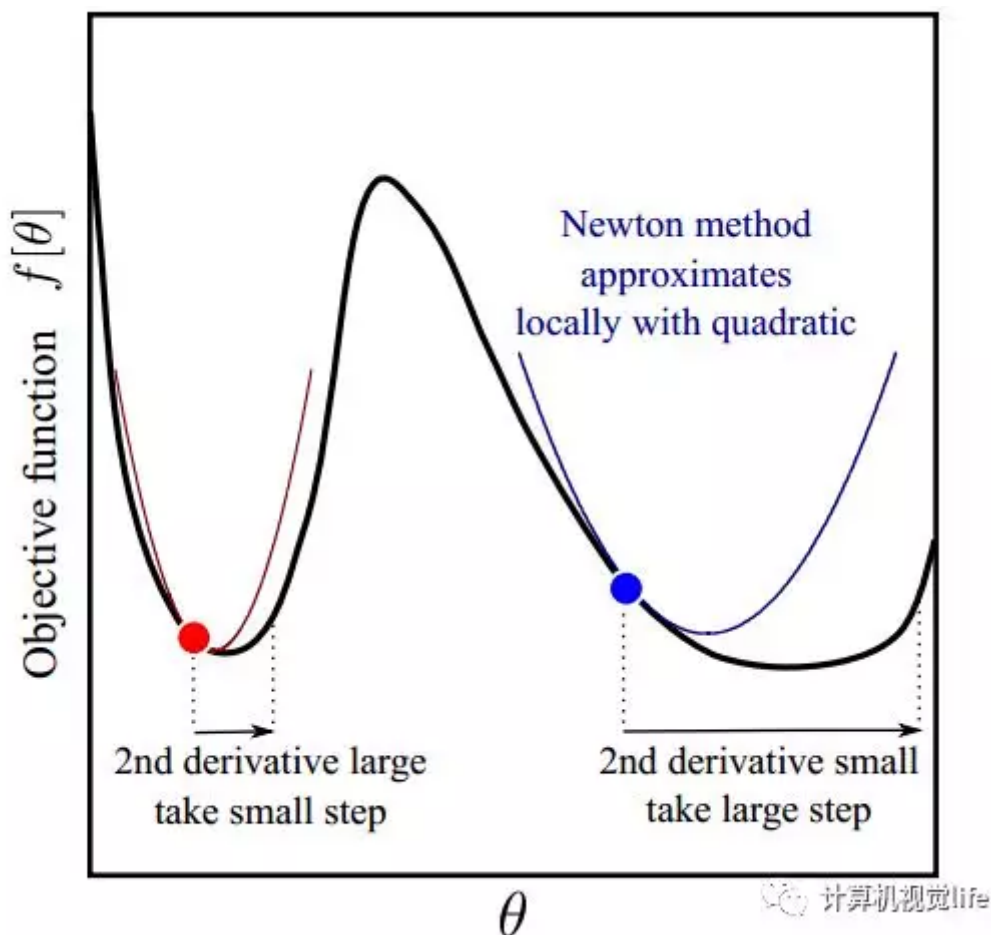
$$h = -H^{-1} * g(x) \quad (29)$$

问题在于，如果H是负定的，且附近有一个局部最大值，牛顿法可能会收敛至该局部最大值。我们可以要求损失函数每一步都是减小的，或者使用如下的混合方法

```

if  $F''(\mathbf{x})$  is positive definite
     $\mathbf{h} := \mathbf{h}_n$ 
else
     $\mathbf{h} := \mathbf{h}_{sd}$ 
 $\mathbf{x} := \mathbf{x} + \alpha \mathbf{h}$ 

```



牛顿法在选择方向时，不仅会考虑坡度是否够大，还会考虑你走了一步之后，坡度是否会变得更大。

由于牛顿法本身需要计算一阶梯度和二阶海森矩阵，并且还需要对 Hessian 求逆，其计算量是非常大的，因此我们希望能通过某种方法近似它。

3.5 高斯-牛顿法

我们来看高斯牛顿法是怎么对牛顿法进行简化的。

将在牛顿法中，我们是将目标函数 $\|f(x + \Delta x)\|^2$ 展开，这里我们考虑直接将函数 $f(x)$ 进行泰勒展开，并忽略高阶项

$$f(x + \Delta x) = f(x) + J(x)h \quad (30)$$

将上式代入我们的目标函数，则问题转化为

$$\underset{\Delta x}{\operatorname{argmin}} \|f(x) + J(x)h\|^2 \quad (31)$$

将目标函数展开，用一个符号来表示它

$$F(h) = \|f(x) + Jh\|^2 = (f(x) + Jh)^T + (f(x) + Jh)$$

$$\begin{aligned} L(h) = \|J(x) + Jh\|^2 &= (J(x) + Jh)^T * (J(x) + Jh) \\ &= \|f(x)\|^2 + 2f(x)^T Jh + h^T J^T Jh \end{aligned} \quad (32)$$

对上式 h 求导有

$$\begin{aligned} L' &= J^T Jh + J^T f(x) \\ L'' &= J^T J \end{aligned} \quad (33)$$

那么根据前面讲到的二阶必要条件， $J^T J$ 需要为半正定的。令一阶导数为 0，则有

$$\begin{aligned} J^T Jh &= -J^T f(x) \\ h &= -(J^T J)^{-1} * (J^T f(x)) \end{aligned} \quad (34)$$

h 是否是下降方向？

那么我们就得到了更新步长的公式

$$h = -(J^T J)^{-1} * (J^T f(x)) \quad (35)$$

高斯牛顿法有效的使用了一阶雅可比矩阵来替代了牛顿法的二阶海森矩阵的求解，降低了计算量，但是其也存在一些问题：

1. 要求 $J^T J$ 正定，这个条件时常会不满足
2. 我们知道泰勒展开本身只对 x 附近的区域才会有效果，一旦步长较大，近似实际上已经失效了

那么怎么解决这个问题？我们引入了 LM 算法。

3.6 LM (Levenberg-Marquardt) 算法

LM 只做了一个很简单的改进，其修正高斯牛顿法的步长为

$$h = -(J^T J + \mu I)^{-1} * (J^T f(x)), \mu > 0 \quad (36)$$

虽然只是简单的改变，但是却带来了几个好处：

1. 对所有的 $\mu > 0$ ， $J^T J + \mu I$ 变成了正定的，从而保证了 h 是梯度下降的方向
2. 当 μ 较大时， $-(J^T J + \mu I)$ 更接近单位阵，整个方法更接近梯度下降法，收敛速度变快
3. 当 μ 较小时， $J^T J + \mu I$ 更接近 $J^T J$ ，整个方法更接近高斯牛顿法

那么如何选取 μ ？初始通常使用如下形式

$$\begin{aligned} \mathbf{A}_0 &= \mathbf{J}(x_0)^T \mathbf{J}(x_0) \\ \mu_0 &= \tau \cdot \max_i \{a_{ii}^0\} \end{aligned} \quad (37)$$

τ 通常自己设定， a_{ii} 为矩阵 A_0 的对角线元素。

其他情况则基于下式来更新：

$$\rho = \frac{F(x) - F(x + h)}{L(0) - L(h)} \quad (38)$$

- 如果上式数值较大，则说明我们的函数 L 是一个好的近似，因此可以考虑降低 μ 来使算法更接近高斯牛顿法
- 如果上式数值较小，说明我们的近似较差，最好减少步长，并且增加 μ 来使用接近梯度下降法

算法的终止条件：

1. 一阶导数为 0： $g(x) = 0$ 或者 $\|g(x)\| < \epsilon_1$ ；
2. x 变化步距离足够小， $\|h\| = \|x_{new} - x\| < \epsilon_x * (\|x\| + \epsilon_2)$ ；
3. 超过最大迭代次数。

算法流程：

Algorithm 3.16. Levenberg–Marquardt method**begin** $k := 0; \quad \nu := 2; \quad \mathbf{x} := \mathbf{x}_0$ $\mathbf{A} := \mathbf{J}(\mathbf{x})^\top \mathbf{J}(\mathbf{x}); \quad \mathbf{g} := \mathbf{J}(\mathbf{x})^\top \mathbf{f}(\mathbf{x})$ $found := (\|\mathbf{g}\|_\infty \leq \varepsilon_1); \quad \mu := \tau * \max\{a_{ii}\}$ **while** (**not** $found$) **and** ($k < k_{\max}$) $k := k+1; \quad \text{Solve } (\mathbf{A} + \mu \mathbf{I}) \mathbf{h}_{\text{lm}} = -\mathbf{g}$ **if** $\|\mathbf{h}_{\text{lm}}\| \leq \varepsilon_2(\|\mathbf{x}\| + \varepsilon_2)$ $found := \mathbf{true}$ **else** $\mathbf{x}_{\text{new}} := \mathbf{x} + \mathbf{h}_{\text{lm}}$ $\varrho := (F(\mathbf{x}) - F(\mathbf{x}_{\text{new}})) / (L(\mathbf{0}) - L(\mathbf{h}_{\text{lm}}))$ **if** $\varrho > 0$ {step acceptable} $\mathbf{x} := \mathbf{x}_{\text{new}}$ $\mathbf{A} := \mathbf{J}(\mathbf{x})^\top \mathbf{J}(\mathbf{x}); \quad \mathbf{g} := \mathbf{J}(\mathbf{x})^\top \mathbf{f}(\mathbf{x})$ $found := (\|\mathbf{g}\|_\infty \leq \varepsilon_1)$ $\mu := \mu * \max\{\frac{1}{3}, 1 - (2\varrho - 1)^3\}; \quad \nu := 2$ **else** $\mu := \mu * \nu; \quad \nu := 2 * \nu$ **end**

LM 算法步长的更新是一种信赖域的方法，这里我们不在详细讲解了，感兴趣的同学可以自行查找相关文献